

Introducción a Git

Control de versiones desde cero

¿Qué es Git?

Git es un sistema de control de versiones: guarda el historial de un proyecto, registra quién hizo cada cambio y cuándo, y permite volver atrás si algo se rompe.

¿Para qué sirve?

- Guarda el historial completo de cambios del proyecto.
- Registra cuándo y quién hizo cada cambio.
- Permite volver a cualquier versión anterior.
- Facilita el trabajo en equipo sin que nadie se pise.

Metáfora: Git es como una máquina del tiempo para tu proyecto. Guardas "puntos de control" y puedes viajar de vuelta a cualquiera de ellos.

¿Por qué necesitamos control de versiones?

Sin Git, cuando programamos suele pasar esto:

- "Antes funcionaba... ¿qué he tocado?"
- "Voy a guardar una copia por si acaso" → final.zip, final_final.zip, ahora_si.zip...
- "He mezclado cambios de varias cosas y ya no sé separarlos."
- "He borrado algo importante sin querer."

Git resuelve todo eso: crea un historial ordenado y recuperable. Sin copias manuales, sin archivos _final_v2.

Además, cuando trabajas en equipo, Git permite que varias personas trabajen sobre el mismo proyecto sin sobrescribirse. Eso lo veremos en el tema de GitHub.

¿Qué es un commit?

Un commit es un punto de guardado del proyecto: una "foto" de cómo estaba todo en ese momento.

Metáfora: guardar partida

- Estás programando (jugando).
- Llegas a un punto estable.
- Guardas la partida → haces un commit.
- Si la lees, vuelves a esa partida.

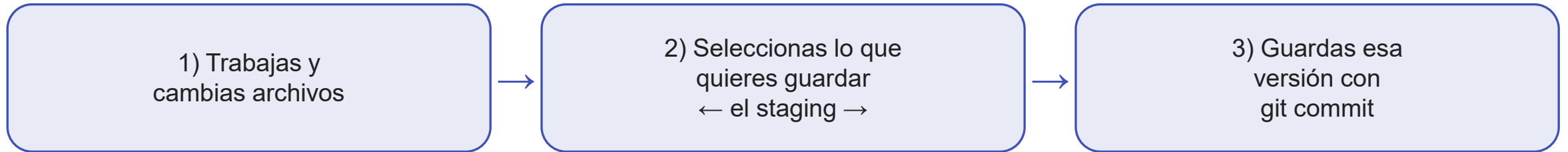
¿Qué lleva un commit?

- Una "foto" del estado de los archivos.
- El autor (nombre y email).
- La fecha y hora.
- Un mensaje descriptivo.

El mensaje de commit debe decir QUÉ has hecho y PARA QUÉ.
Ejemplos buenos: "Añade lista inicial de libros" · "Corrige cálculo del total con descuento"

La idea que más cuesta: el staging (la bandeja)

Git no funciona como un «Guardar» de Word. Antes del commit hay un paso intermedio:

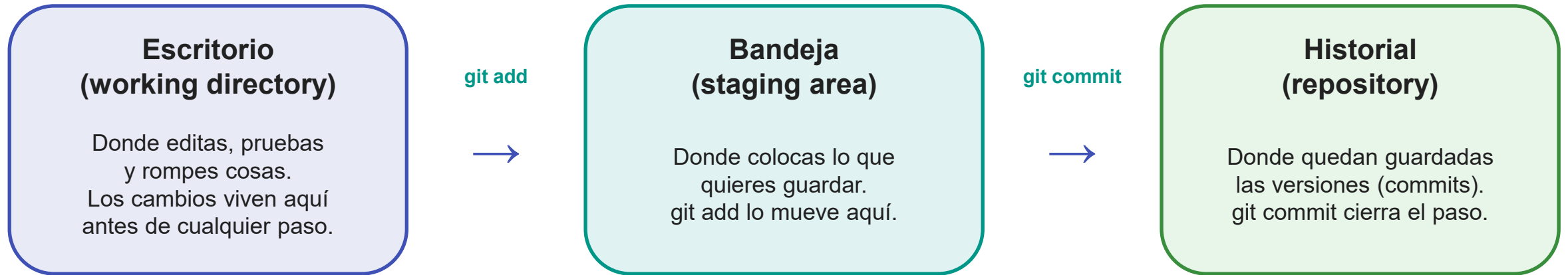


Staging area (también llamado índice o bandeja)

- Lo que está en staging → "esto SÍ va al próximo commit".
- Lo que NO está en staging → "esto todavía no".
- **El commit guarda lo que hayas puesto en staging, no todo lo que hay en la carpeta.**

git add = "pon esto en la bandeja" git commit = "pega lo de la bandeja en el historial"

Staging — las tres zonas de Git



¿Por qué existe la bandeja?

Porque los cambios se mezclan. Mientras arreglas un bug, también tocas el README y un CSS. Con staging puedes hacer un commit para cada cosa y el historial queda limpio.

"Ahora guardo solo el bug. Después el README. Después el CSS."

Staging — commits limpios: uno por idea

Supón que has tocado 3 archivos: Pedido.java (bug) · README.md (texto) · styles.css (estilo).

✗ Lo que NO interesa

```
git add .  
git commit -m "Cambios"
```

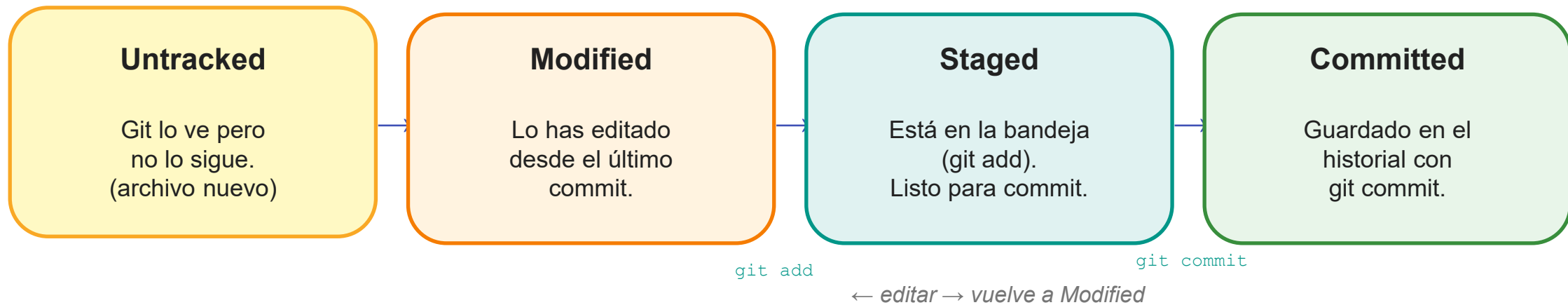
✓ Lo recomendable

```
git add Pedido.java  
git commit -m "Corrige bug en cálculo"  
  
git add README.md  
git commit -m "Actualiza instrucciones"  
  
git add styles.css  
git commit -m "Mejora estilos del layout"
```

Historial claro, más fácil deshacer por partes, revisiones más rápidas.

git add . mete en staging TODO lo nuevo/modificado. Al principio, mejor añadir archivo a archivo.

Estados de un archivo en Git



Ejemplo rápido

- Creas hola.txt → Untracked
- `git add hola.txt` → Staged
- `git commit ...` → Committed
- Editas hola.txt → Modified (hay que volver a hacer git add)

Instalación de Git

Antes de nada: comprueba si ya lo tienes.

```
git --version
```

- Si devuelve "git version 2.4x.x" → ya lo tienes, para clase te vale.
- Si sale "command not found" → hay que instalarlo.

Por sistema operativo

Sistema	Cómo instalarlo
Windows	Instala Git for Windows (git-scm.com/install/windows). Incluye Git Bash — úsala al principio.
macOS	Ejecuta <code>git --version</code> . Si macOS pide instalar Command Line Tools, acepta y espera.
Linux (Debian/Ubuntu)	<code>sudo apt update && sudo apt install git</code>

Configuración básica (una sola vez)

Git guarda en cada commit quién lo hizo. Sin esto, Git se quejará con "Please tell me who you are".

1) Nombre y email

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tuemail@ejemplo.com"  
  
# Comprobar que ha quedado bien:  
git config --global --list
```

2) Rama por defecto: main

```
git config --global init.defaultBranch main
```

3) Editor por defecto (recomendado con VS Code)

```
git config --global core.editor "code --wait"
```

Estas configuraciones se guardan en ~/.gitconfig y se aplican a todos los repos del ordenador.

Crear un repositorio: git init

Para que Git empiece a guardar el historial de un proyecto, necesitas inicializar un repositorio: una carpeta «con Git activado».

Pasos

```
cd ~/Documents/pruebas_git    # Entra en la carpeta del proyecto
git init                      # Inicializa Git
git status                     # Comprueba que está listo
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git
• $ git init
Initialized empty Git repository in C:/Users/Aitor/Documents/pruebas_git/.git/

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
• $ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
○ $
```

La carpeta .git/ es la memoria del repositorio. NO la borres ni edites a mano: si la eliminas, el proyecto pierde todo el historial.

git status — ¿qué está pasando?

git status te dice en qué situación está el repo y en qué estado están tus archivos. Úsalo siempre antes de git add y git commit.

```
git status
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
$ ls
adios.txt hasta-luego.txt hola.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   adios.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hasta-luego.txt
```

- **Changes to be committed:**
 - → está en staging, listo para el próximo commit.
- **Untracked files:**
 - → Git lo ve, pero aún no lo sigue (hay que hacer git add).

git diff — ¿qué ha cambiado exactamente?

git diff muestra los cambios línea a línea respecto al último commit. Útil para revisar qué has tocado antes de guardar. No cambia nada.

```
git diff
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
$ git diff
diff --git a/hola.txt b/hola.txt
index e69de29..bf27e94 100644
--- a/hola.txt
+++ b/hola.txt
@@ -0,0 +1 @@
+esta línea es nueva, no estaba cuando he hecho el commit antes.
\ No newline at end of file
```

- Las líneas con + son lo nuevo que has añadido.
- Las líneas con — son lo que has eliminado o reemplazado.
- Mientras solo ejecutas git diff, el proyecto no cambia.

git add — pongo esto en la bandeja (staging)

git add no guarda nada todavía: solo prepara archivos para el próximo commit.

```
git add adios.txt           # Un archivo concreto
git add adios.txt hola.txt  # Varios a la vez
git add .                   # Todo lo nuevo/modificado (con cuidado)
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   adios.txt
    modified:   hola.txt

Untracked files:
  (use "git add <file>..." to include in what will be committed)
    hasta-luego.txt
```

- Changes to be committed: new file adios.txt, modified hola.txt → están en staging.
- Untracked files: hasta-luego.txt → sigue fuera porque no has hecho git add.

git add . mete en staging TODO lo nuevo/modificado. Al principio, mejor añadir archivo a archivo.

git commit — guardo la partida

git commit guarda en el historial lo que esté en staging. Solo lo que tú has puesto con git add.

```
git commit -m "Añado los archivos hola y adios"
```

```
# Para corregir el mensaje del último commit (antes de subir):  
git commit --amend -m "Mensaje corregido"
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)  
• $ git commit -m "Añado los archivos hola y adios"  
[main 37d99c1] Añado los archivos hola y adios  
 2 files changed, 1 insertion(+)  
 create mode 100644 adios.txt  
  
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)  
• $ git status  
On branch main  
Untracked files:  
  (use "git add <file>..." to include in what will be committed)  
      hasta-luego.txt  
  
nothing added to commit but untracked files present (use "git add" to track)
```

- Git confirma el commit: hash corto y mensaje.
- 2 files changed → los archivos que estaban en staging.
- **hasta-luego.txt sigue en Untracked — no se guardó porque NO estaba en staging.**

git log — el historial de commits

```
# Historial completo
git log

# Una línea por commit (el más usado)
git log --oneline

# Con gráfico de ramas
git log --oneline --graph
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
$ git log
commit 37d99c17f1219630d5426466f3811f00fc41c5f3 (HEAD -> main)
Author: aitor <aitor[REDACTED]@gmail.com>
Date:   Sun Feb 1 18:23:11 2026 +0100

    Añado los archivos hola y adios

commit bf0b67188cc56df9fc6d9ceae94f4192ade3eeeb
Author: aitor <aitor[REDACTED]@gmail.com>
Date:   Sun Feb 1 18:19:30 2026 +0100

    creando hola.txt
```

HEAD → main: estás en la rama main y HEAD apunta al último commit.

El hash: identificador único de cada commit

Cada commit tiene un hash largo (a3f8c21d...). No hace falta escribirlo entero: con los 7 primeros caracteres es suficiente. Esos son los que usarás en git show, git revert y git reset.

git log --oneline — una línea por commit

```
git log --oneline
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git_libros (main)
```

```
$ cd ~/Documents/pruebas_git_libros
```

- ```
git log --oneline
```

```
1cd8f5a (HEAD -> main) Elimina un llibre de la llista
```

```
448751c Afegeix el quart llibre
```

```
58fbc71 Primer commit – llista inicial de llibres
```

Los 7 caracteres del inicio de cada línea (1cd8f5a, 448751c...) son el hash abreviado. Los usarás en git show, git revert y git reset.

HEAD → main: estás en la rama main y en el commit más reciente.

# git restore — descartar cambios de un archivo

Si modificas un archivo y no quieres esos cambios, puedes volver al estado del último commit.

```
git restore hola.txt
```

```
$ git diff
diff --git a/hola.txt b/hola.txt
index bf27e94..ad3b0c7 100644
--- a/hola.txt
+++ b/hola.txt
@@ -1,3 @@
-esta línea es nueva, no estaba cuando he hecho el commit antes.
\ No newline at end of file
+esta línea es nueva, no estaba cuando he hecho el commit antes.
+
+hago cambios!!!! y luego borraré esta línea
\ No newline at end of file

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
$ git restore hola.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
$ git diff
```

- Primero git diff muestra las líneas nuevas (+).
- Después git restore hola.txt.
- Al volver a ejecutar git diff, no sale nada → el archivo ha vuelto al estado del último commit.

CUIDADO: git restore borra cambios no guardados en commits. Si te importan, haz commit antes.

# git restore --staged — sacar del staging sin borrar cambios

Sirve cuando has hecho git add por error: saca el archivo del staging, pero los cambios en el archivo se mantienen.

```
git add hasta-luego.txt # Lo metes en staging
git restore --staged hasta-luego.txt # Te arrepientes y lo sacas
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
• $ git add hasta-luego.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
• $ git status
On branch main
Changes to be committed:
 (use "git restore --staged <file>..." to unstage)
 new file: hasta-luego.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
• $ git restore --staged hasta-luego.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git (main)
• $ git status
On branch main
Untracked files:
 (use "git add <file>..." to include in what will be committed)
 hasta-luego.txt

nothing added to commit but untracked files present (use "git add" to track)
```

Truco mental:  restore --staged = "saca de la bandeja"    restore (sin --staged) = "deshaz cambios del archivo"

# Volver a una versión anterior — ¿qué quiero exactamente?

Antes de actuar, hazte esta pregunta:

¿Qué necesitas?

Ver qué cambió  
un commit

`git show <hash>`

Recuperar un  
archivo concreto

`git restore --source=  
<hash> archivo`

Deshacer un  
commit entero

¿Ya lo has subido  
a GitHub?

**SÍ** → `git revert` (conserva historial)

**NO** → `git reset` (puede borrar  
historial)

```
git log --oneline # Siempre el primer paso: ver qué commits tienes y sus hashes
```

# Ver qué cambió un commit: git show

Antes de deshacer nada, conviene ver exactamente qué hizo ese commit. Mientras solo ejecutas git show, no cambias nada en el proyecto.

```
git show 58fbc71
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git_libros (main)
$ git show 58fbc71
commit 58fbc71f7736d6a3a852a1e3e0f3eb80a3421c2a
Author: aitor <aitorventura7@gmail.com>
Date: Fri Jun 26 21:03:48 2026 +0200

 Primer commit – llista inicial de llibres

diff --git a/llibres.txt b/llibres.txt
new file mode 100644
index 0000000..846aadf
--- /dev/null
+++ b/llibres.txt
@@ -0,0 +1,3 @@
+El senyor dels anells
+1984
+Dune
```

- Las líneas con + son lo que ese commit añadió.
- Las líneas con — son lo que eliminó.
- Solo es una consulta — el proyecto no cambia.

# Opción A — Recuperar un archivo a como estaba antes

Útil cuando un archivo concreto ha empeorado y quieres recuperarlo a una versión anterior, sin tocar el resto del proyecto ni el historial.

```
git restore --source=58fbc71 llibres.txt
cat llibres.txt # Comprueba el resultado
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git_libros (main)
• $ git restore --source=58fbc71 llibres.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git_libros (main)
• $ cat llibres.txt
El senyor dels anells
1984
Dune
```

- El archivo queda exactamente como estaba en ese commit.
- `git status` lo muestra como modified — el historial NO ha cambiado.
- **Para guardarlo definitivamente: `git add` + `git commit` después.**
- Esto NO deshace el commit — solo trae una versión antigua del archivo a tu carpeta.

## Opción B — Deshacer un commit completo: git revert

git revert crea un commit nuevo que deshace exactamente los cambios de un commit anterior. No borra nada del historial: es la opción más segura.

```
git revert 1cd8f5a
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git_libros (main)
• $ git revert --no-edit 1cd8f5a
[main c22827f] Revert "Elimina un llibre de la llista"
Date: Fri Jun 26 21:09:19 2026 +0200
1 file changed, 1 insertion(+)

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_git_libros (main)
• $ git log --oneline
c22827f (HEAD -> main) Revert "Elimina un llibre de la llista"
1cd8f5a Elimina un llibre de la llista
448751c Afegeix el quart llibre
58fbc71 Primer commit – llista inicial de llibres
```

- Git genera el mensaje del commit de reversión automáticamente.
- El commit original sigue en el historial, neutralizado.
- **Úsalo siempre que hayas subido el repo a GitHub o haya más personas. No reescribe el historial.**

## Opción C — git reset: situación de partida

git reset mueve HEAD hacia atrás en el historial. Más potente que revert: borra commits como si nunca hubieran existido. Solo para repos locales que nadie más ha visto.

Los tres ejemplos (--soft, --mixed, --hard) parten del mismo repo con estos 3 commits:

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_soft (main)
● $ git log --oneline
1d1081f (HEAD -> main) Corregeix majuscles del titol
dc7c583 Afegeix el quart llibre
0660c45 Primer commit – llista inicial de llibres
```

El commit más reciente («Corregeix majuscles del titol») es el que va a desaparecer con el reset. Lo que cambia entre las tres variantes: ¿adónde van sus cambios?



# git reset --soft — cambios en staging

El commit desaparece, pero Git deja los cambios ya preparados para el siguiente commit — como si ya hubieras hecho git add. Solo falta git commit.

```
git reset --soft HEAD~1
```

```
A continuación puedes hacer directamente:
git commit -m "mensaje correcto"
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_soft (main)
● $ git reset --soft HEAD~1

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_soft (main)
● $ git log --oneline
dc7c583 (HEAD -> main) Afegeix el quart llibre
0660c45 Primer commit – llista inicial de llibres

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_soft (main)
● $ git status --short
M llibres.txt
```

git status --short muestra "M llibres.txt" — la M está en la zona de staging. No hace falta git add.

# git reset --mixed — cambios en el escritorio (opción por defecto)

El commit desaparece y los cambios vuelven al archivo, pero Git NO los prepara para el siguiente commit. Es como si nunca hubieras hecho git add.

```
git reset HEAD~1 # --mixed es el comportamiento por defecto
```

```
A continuación necesitas:
git add llibres.txt
git commit -m "mensaje"
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_mixed (main)
• $ git reset HEAD~1
Unstaged changes after reset:
M llibres.txt

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_mixed (main)
• $ git log --oneline
c1d8ebd (HEAD -> main) Afegeix el quart llibre
3dbde60 Primer commit – llista inicial de llibres

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_mixed (main)
• $ git status --short
M llibres.txt
```

- **Git avisa: "Unstaged changes after reset".**
- git status --short muestra " M llibres.txt" — la M está en la zona de trabajo (fuera de staging).
- **Diferencia con --soft: necesitas git add antes de poder hacer commit. Con --soft no.**

# git reset --hard — cambios borrados definitivamente

El commit desaparece Y los cambios se borran definitivamente. El proyecto vuelve exactamente al estado del commit anterior. Sin vuelta atrás.

```
git reset --hard HEAD~1
Git responde: HEAD is now at <hash> <mensaje>
```

```
Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_hard (main)
• $ git reset --hard HEAD~1
 HEAD is now at 2fef65d Afegeix el quart llibre

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_hard (main)
• $ git log --oneline
 2fef65d (HEAD -> main) Afegeix el quart llibre
 68edcba Primer commit – llista inicial de llibres

Aitor@DESKTOP-5AC8HPL MINGW64 ~/Documents/pruebas_hard (main)
• $ cat llibres.txt
 El senyor dels anells
 1984
 Dune
 El nom de la rosa
```

- El log pasa de 3 a 2 commits.
- cat llibres.txt muestra el archivo sin la corrección de mayúsculas — ese cambio ha desaparecido.
- **No hay papelera ni deshacer. Úsalo solo si estás completamente seguro.**

# git reset — comparativa de las tres variantes

La única pregunta que importa: ¿adónde van los cambios del commit eliminado?

| Variante       | ¿Adónde van los cambios?     | git status muestra               | Siguiente paso          | ¿Recuperable? |
|----------------|------------------------------|----------------------------------|-------------------------|---------------|
| <b>--soft</b>  | Staging (listos para commit) | M archivo<br>(M en zona staging) | git commit directamente | Sí            |
| <b>--mixed</b> | Escritorio de trabajo        | M archivo<br>(M en zona trabajo) | git add + git commit    | Sí            |
| <b>--hard</b>  | Se borran definitivamente    | (nada)                           | —                       | <b>NO</b>     |

Regla práctica: si el repo solo es tuyo y es local → git reset. Si ya lo has subido o hay más gente → git revert.

# .gitignore — archivos que no queremos guardar

El archivo .gitignore le dice a Git qué archivos o carpetas debe ignorar aunque estén en la carpeta del proyecto.

```
Archivos de log
*.log

Compilación Java
/target/
/out/
*.class

IntelliJ
.idea/
*.iml

Credenciales — NUNCA subir
.env
secrets.txt
```

## Sintaxis de patrones

- \*.log → cualquier archivo que acabe en .log
- /tmp/ → carpeta tmp en la raíz del repo
- .idea/ → carpeta de configuración de IntelliJ
- secrets.txt → archivo concreto por nombre

Solo afecta a lo que aún no está en el historial. Si ya fue commiteado, `git rm --cached <archivo>` es necesario.

gitignore.io genera un .gitignore completo para tu tipo de proyecto (Java, Node, Python...) con un clic.

# Referencia rápida — todos los comandos esenciales

| Situación                                       | Comando                               |
|-------------------------------------------------|---------------------------------------|
| Inicializar un repositorio                      | git init                              |
| Ver el estado del repo y los archivos           | git status                            |
| Ver qué cambió línea a línea                    | git diff                              |
| Añadir un archivo al staging                    | git add <archivo>                     |
| Guardar un commit                               | git commit -m "mensaje"               |
| Ver el historial (resumido)                     | git log --oneline                     |
| Descartar cambios de un archivo                 | git restore <archivo>                 |
| Sacar un archivo del staging                    | git restore --staged <archivo>        |
| Ver qué hizo un commit concreto                 | git show <hash>                       |
| Recuperar un archivo a una versión anterior     | git restore --source=<hash> <archivo> |
| Deshacer un commit (seguro, conserva historial) | git revert <hash>                     |
| Deshacer commit, cambios en staging             | git reset --soft HEAD~1               |
| Deshacer commit, cambios en escritorio          | git reset HEAD~1                      |
| Deshacer commit y borrar cambios                | git reset --hard HEAD~1               |